May 15th , 2026

**Database system**
**SWEG2108**

College of Electrical and Mechanical Engineering
Department of Software Engineering

# ELECTRONIC VOTING SYSTEM : PROJECT REPORT

| GROUP MEMBERS | ID |
|---|---|
| 1.ABEL ALEM | ETS0022/17 |
| 2. ALAZAR WALLE | ETS 0127/17 |
| 3. AYMEN NIGUSSIE | ETS 0211/17 |
| 4. BIRUK ABRHAM | ETS 0324/17 |
| 5. DAWIT AREGAWI | ETS 0418/17 |
| 6. EDILAWIT SAFEWORK | ETS0464/17 |

## SECURE LINK – AUTOMATED ELECTRONIC VOTING SYSTEM

Submitted To: Dr. Yaynshet Medhin Addis Ababa, Ethiopia

# CHAPTER ONE: INTRODUCTION

## 1.1 Background of the Organization

National Election Board of Ethiopia is the governmental organization responsible for organizing, managing, and supervising elections in Ethiopia. The organization ensures that elections are conducted fairly, transparently, and according to the constitution and electoral laws of the country. Its responsibilities include voter registration, candidate registration, election scheduling, vote collection, result management, and announcement of election outcomes.

Currently, many election activities rely heavily on manual or semi-manual processes. These traditional methods can consume a large amount of time, require extensive human resources, and may increase the possibility of errors, delays, and security problems. As the number of voters continues to grow, there is a need for a modern computerized system that can improve efficiency, accuracy, transparency, and security in the election process.

To address these challenges, the proposed Electronic Voting System is designed to help the National Election Board of Ethiopia manage election activities digitally. The system will provide secure voter management, electronic vote casting, automatic vote counting, and fast result generation while reducing paperwork and operational complexity.

## 1.2 Problem Statement

The current election management process used by the National Election Board of Ethiopia faces several challenges due to its dependence on manual procedures and paper-based operations. These challenges negatively affect the efficiency, transparency, and reliability of the election process.

Some of the major problems include:

➢ Long waiting times during voter registration and voting processes.

➢ Human errors during vote counting and result compilation.

➢ Difficulty in managing large amounts of voter data manually.

➢ Risk of duplicate voting and voter fraud.

➢ Delays in announcing election results.

➢ High operational costs due to paper usage and manual labor.

➢ Limited data security and difficulty in maintaining accurate records.

➢ Difficulty in generating fast and reliable election reports.

### 1.3 Project Motivation

The integrity and efficiency of any democratic process rest heavily on the credibility of its electoral system. The National Election Board of Ethiopia (NEBE), as the constitutional body mandated to oversee free and fair elections, faces increasing pressure from a growing electorate and rising expectations for transparent governance. However, the Board's current reliance on manual, paper-based processes presents significant barriers to fulfilling this mandate effectively.

This project is motivated by the urgent need to overcome the critical limitations of the existing manual electoral system. The current approach leads to excessively long waiting times for voters, exposes the process to human error during vote counting and result compilation, and creates significant challenges in securely managing vast amounts of voter data. These inefficiencies not only delay the announcement of results but also strain operational budgets due to high paper consumption and extensive manual labor.

Furthermore, the manual system introduces vulnerabilities that threaten the fundamental principles of a free election. Risks of duplicate voting, voter fraud, and limited data security undermine public trust. The inability to generate fast, reliable, and auditable election reports further hampers transparency and the ability to resolve disputes promptly.

Therefore, this project is motivated by the imperative to modernize Ethiopia's electoral process through a secure, computerized Electronic Voting System. By automating voter management, electronic vote casting, and real-time result generation, the system aims to:

Enhance Efficiency by reducing long queues and accelerating result tabulation.

Improve Accuracy by eliminating human errors in counting and data handling.

Strengthen Security by minimizing fraud and protecting voter data.

Increase Transparency by enabling faster, verifiable reporting.

Reduce Operational Costs by decreasing reliance on paper and extensive manual labor.

In essence, this project is driven by the need to restore and reinforce public confidence in the electoral process, ensuring that elections in Ethiopia are not only free and fair but also demonstrably efficient, accurate, and secure in the digital age.

## 1.4 Objectives of the Study

General Objective

To design and develop a secure, reliable, and efficient Electronic Voting System for the

National Election Board of Ethiopia that improves election management and ensures

accurate and transparent voting processes.

Specific Objectives

The specific objectives of the system are:

✓ To create a database system for storing and managing voter information securely.

✓ To automate the voter registration process.

✓ To enable secure electronic vote casting for registered voters.

✓ To prevent duplicate voting and unauthorized access.

✓ To automate vote counting and result calculation.

✓ To generate accurate election reports quickly.

✓ To improve transparency and efficiency in the election process.

✓ To reduce paperwork and manual operational costs.

✓ To maintain the confidentiality and integrity of election data.

✓ To provide backup and recovery mechanisms for election records.

## 1.5 Scope and Delimitations

Scope of the System

The scope of the proposed Electronic Voting System defines the functional boundaries and features that will be implemented based on the database design. The system will cover the following areas:

**1. Voter Management**

- Registration of voters with personal details (full name, date of birth, gender, address, national ID, phone, email).
- Unique identification of voters via national_id, phone_number, and email constraints.
- Tracking voter status (active/inactive) and registration date.
- Association of voters with their respective region (e.g., Addis Ababa, Oromia, Amhara).

**2. Candidate and Party Management**

- Registration of political parties with name, abbreviation, registration number, symbol URL, and description.
- Registration of candidates with personal details, photo URL, status, and party affiliation.
- Ability to link candidates to their respective parties.

**3. Polling Station Management**

- Definition of polling stations with name, unique station code, address, and geographic coordinates (latitude/longitude).
- Assignment of each polling station to a specific region.

**4. Voting Process**

- Recording of each vote cast, including which voter voted for which candidate, at which polling station, and on what date.
- Prevention of duplicate voting through the UNIQUE constraint on voter_id within the vote table (each voter can cast only one vote).
- Validation of votes via the is_valid flag (e.g., marking votes as valid or invalid).

**5. Result Management and Reporting**

- Automatic vote counting and aggregation using SQL queries (e.g., total vote.sql).
- Generation of election results showing:

- Candidate name
- Party name
- Total votes received (including candidates with zero votes via LEFT JOIN).
- Sorting of results in descending order of votes (winner at the top).

**6. Administrative Functions**

- Management of system administrators with unique usernames and emails.
- Role-based access control (via role field, e.g., manager).
- Tracking of admin last login and account creation date.

Limitations of the System

- **No biometric or multi-factor authentication** – The system relies solely on basic identifiers (national ID, phone, email) and username/password for admins, lacking fingerprint, facial recognition, or two-factor authentication to prevent identity fraud.
- **No online or remote voting capability** – Voting is restricted to physical polling stations with manual data entry. The system does not support internet-based voting, which limits accessibility for voters unable to travel.
- **No audit trail or tamper-proof security** – The system lacks cryptographic audit logs, blockchain integration, or mechanisms to track who entered or modified data, making it vulnerable to undetected internal alterations.
- **No integration with external government systems** – Voter data cannot be automatically verified against national ID databases, civil registries, or other official sources, increasing the risk of unverified registrations.
- **No support for multiple election types or dispute resolution** – The system handles only a single election cycle without provisions for primary elections, run-offs, referendums, or built-in recount and dispute resolution workflows.
- **No voter analytics or turnout reporting** – While the system counts votes per candidate, it cannot calculate voter turnout percentages, demographic breakdowns, or generate historical trend analyses essential for electoral assessment.

## 1.6 Significance of the Study

This study is significant because it addresses the critical inefficiencies of Ethiopia's manual electoral process through a secure, automated Electronic Voting System. The proposed system offers the following key benefits:

Reduces operational costs by minimizing paper usage, printing, and manual labor.

Eliminates human errors in vote counting and result compilation through automated aggregation.

Prevents duplicate voting via database-enforced unique constraints on voter IDs.

Generates instant results using SQL queries, reducing post-election delays and tensions.

Enhances data security through primary keys, foreign key constraints, and structured relational design.

Furthermore, the system strengthens democratic governance in Ethiopia by improving transparency and public trust in election outcomes. It is scalable to accommodate a growing electorate, provides a foundation for future enhancements such as biometric integration or mobile voting, and supports the National Election Board in fulfilling its constitutional mandate to conduct free, fair, and credible elections.

### 1.7 Methodology

The proposed Electronic Voting System will be developed following a structured approach to ensure reliability, security, and alignment with the National Election Board of Ethiopia's requirements. The methodology includes the following key phases and techniques:

Development Approach: Waterfall model, consisting of requirements gathering, system design, implementation, testing, deployment, and maintenance phases.

Database Design: Relational database using MySQL, with tables for region, voter, party, candidate, polling_station, vote, and admin, linked via primary and foreign keys to ensure data integrity.

Backend Technology: PHP or Python (Django/Flask) to handle business logic, authentication, and database interactions.

Frontend Technology: HTML, CSS, and JavaScript for a web-based administrative interface accessible to election officials.

Security Measures: Password hashing for admin accounts, input validation to prevent SQL injection, and unique constraints to prevent duplicate voter registrations and votes.

Vote Counting Mechanism: Automated aggregation using SQL queries (e.g., total vote.sql) with LEFT JOIN to include candidates receiving zero votes.

Testing Strategy: Unit testing for database operations, integration testing for vote casting workflows, and user acceptance testing with mock election data.

Deployment: Local server environment within the National Election Board's secure infrastructure, with regular database backups.

### 1.8 Tools and Technologies Used

The following tools and technologies are used in this project:

- MySQL: For relational database implementation and structured data management
- MongoDB: For NoSQL database and flexible data storage
- Draw.io: For designing ER diagrams
- GitHub: For version control and collaboration
- Google Docs: For documentation and report preparation

# CHAPTER 2: DATABASE DESIGN

The database design chapter presents the structural foundation of the Electronic Voting System developed for the National Election Board of Ethiopia. A well-structured database is essential to any information system, as it determines how data is organized, stored, related, and retrieved. In this project, the database serves as the backbone of the entire voting system, managing critical information related to voters, candidates, political parties, polling stations, votes, regions, and system administrators.

The Electronic Voting System employs a dual-database approach, utilizing both a relational database management system (MySQL) and a NoSQL database (MongoDB). The relational database is designed to enforce data integrity, prevent duplicate voting, and maintain structured relationships between entities through the use of primary keys, foreign keys, and unique constraints. MongoDB, on the other hand, provides flexibility in data storage and supports document-oriented representations of the same entities, offering an alternative data management perspective.

This chapter covers the conceptual, logical, and physical design of the database. It presents the Entity-Relationship Diagram (ERD) that illustrates the entities involved and their relationships, followed by the logical table structure showing attributes and data types, and finally the physical implementation through SQL and MongoDB scripts. Together, these design layers ensure that the system can accurately record and process election data while maintaining the confidentiality, integrity, and availability of all stored information.

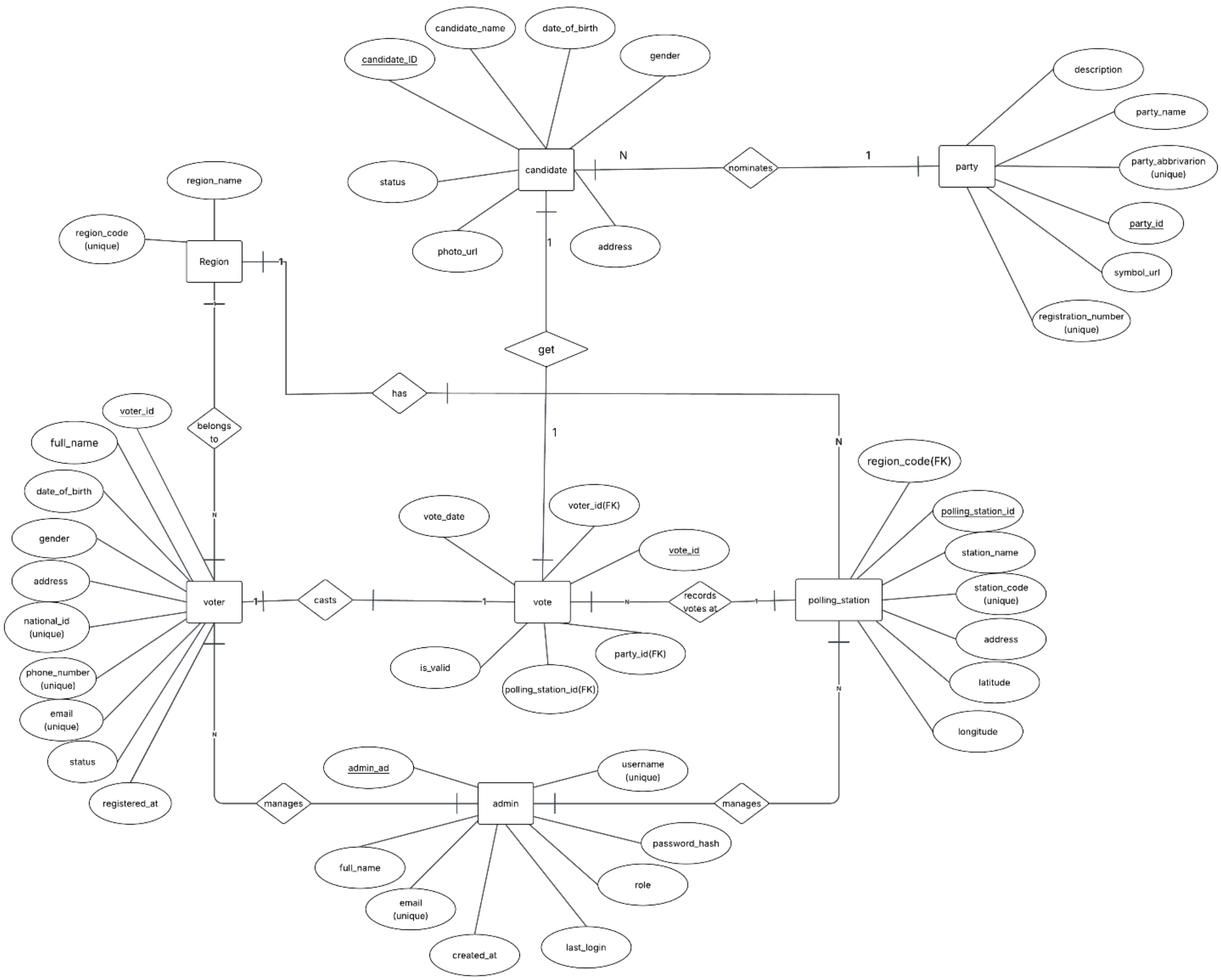
## 2.1 Entity Relationship (ER) Diagram Description

An Entity-Relationship (ER) diagram is a conceptual representation of the data and the relationships between different data entities within a system. It provides a visual blueprint of how the database is structured before the actual implementation, helping to identify the key entities, their attributes, and the associations that exist among them. In the context of the Electronic Voting System, the ER diagram serves as the foundation upon which the entire database is built, ensuring that all necessary data requirements are captured and logically organized.

The ER diagram of the Electronic Voting System consists of seven core entities: Region, Voter, Party, Candidate, Polling Station, Vote, and Admin. Each entity represents a distinct real-world object or concept relevant to the election process, and each is defined by a set of attributes that describe its characteristics. The relationships between these entities reflect the real-world interactions that occur during an election, such as a voter casting a vote, a candidate being nominated by a party, or a polling station being assigned to a specific region.

The diagram follows standard ER notation, where rectangles represent entities, ellipses represent attributes, diamonds represent relationships, and lines indicate the connections between them. Underlined attributes denote primary keys, while attributes marked as unique indicate fields that must hold distinct values across all records. The cardinality notations on the relationship lines — such as one-to-one (1:1), one-to-many (1:N), and many-to-one (N:1) — define the nature and constraints of each relationship between entities.

The following ER diagram illustrates the complete conceptual design of the Electronic Voting System database:



## 2.2 Description of Major Entities

### 2.2.1 Region Entity

The Region entity stores information about the geographical administrative areas within Ethiopia where voters are registered and polling stations are located.

**Attributes of Region**

| Attribute | Description |
|---|---|
| **region_code** | Unique identifier for each region (Primary Key) |
| **region_name** | The name of the region (e.g., Addis Ababa, Oromia, Amhara) |

**Purpose of the Entity**

This entity serves as a reference table that organizes election data by geographic location. It enables the system to associate voters and polling stations with their respective regions.

**Relationships**

- One region can have many voters registered under it.
- One region can have many polling stations assigned to it.

### 2.2.2 Voter Entity

The Voter entity stores the personal information of all individuals who are eligible to participate in the election.

**Attributes of Voter**

| Attribute | Description |
|---|---|
| **voter_id** | Unique identifier for each voter, auto-generated by the system (Primary Key) |
| **full_name** | The complete name of the voter |
| **date_of_birth** | The voter's date of birth |
| **gender** | The gender of the voter |
| **address** | The residential address of the voter |

| | |
|---|---|
| **national_id** | The national identification number of the voter (Unique) |
| **phone_number** | The voter's contact phone number (Unique) |
| **email** | The voter's email address (Unique) |
| **status** | Indicates whether the voter's account is active or inactive |
| **registered_at** | The date on which the voter was registered |
| **region_code** | Foreign key linking the voter to their respective region |

**Purpose of the Entity**

This entity serves as the primary user management component of the voting system. It enables voter identification, registration tracking, and prevention of duplicate registrations through unique constraints on national_id, phone_number, and email.

**Relationships**

- One voter belongs to one region.
- One voter can cast only one vote (enforced by UNIQUE constraint on voter_id in the vote table).

### 2.2.3 Party Entity

The Party entity stores information about the political parties that are registered and participating in the election.

**Attributes of Party**

| Attribute | Description |
|---|---|
| **party_id** | Unique identifier for each political party (Primary Key) |
| **party_name** | The full official name of the political party |
| **abbreviation** | The shortened name or acronym of the party (Unique) |
| **registration_number** | The official registration number of the party (Unique) |
| **symbol_url** | The file path or URL of the party's symbol or logo image |
| **description** | A brief description of the party and its background |

**Purpose of the Entity**

This entity serves as the reference table for all registered political parties in the election. It enables the system to associate candidates with their respective parties and display party information in election results.

**Relationships**

- One party can nominate many candidates.
- One party can receive many votes through its nominated candidates.

### 2.2.4 Candidate Entity

The Candidate entity stores the personal and electoral details of all individuals who are contesting in the election.

**Attributes of Candidate**

| Attribute | Description |
|---|---|
| **candidate_id** | Unique identifier for each candidate (Primary Key) |
| **candidate_name** | The full name of the candidate |
| **date_of_birth** | The candidate's date of birth |
| **gender** | The gender of the candidate |
| **address** | The residential address of the candidate |
| **photo_url** | The file path or URL of the candidate's profile photo |
| **status** | Indicates whether the candidate is active or inactive in the election |
| **party_id** | Foreign key linking the candidate to their political party |

**Purpose of the Entity**

This entity serves as the central component for managing election contestants. It enables the system to display candidate information, associate candidates with their parties, and track the votes received by each candidate.

**Relationships**

- One candidate belongs to one political party.
- One candidate can receive many votes.

2.2.5 Polling Station Entity

The Polling Station entity stores information about the physical locations where voters cast their votes during the election.

**Attributes of Polling Station**

| Attribute | Description |
|---|---|
| **polling_station_id** | Unique identifier for each polling station (Primary Key) |
| **station_name** | The name of the polling station |
| **station_code** | A unique code assigned to identify the polling station (Unique) |
| **address** | The physical address of the polling station |
| **latitude** | The geographic latitude coordinate of the polling station |
| **longitude** | The geographic longitude coordinate of the polling station |
| **region_code** | Foreign key linking the polling station to its respective region |

**Purpose of the Entity**

This entity serves as the location management component of the voting system. It enables the system to track where each vote was cast and associate polling stations with their respective regions for geographic reporting.

**Relationships**

- One polling station belongs to one region.
- One polling station can record many votes.

---

**2.2.6 Vote Entity**

The Vote entity records every vote cast during the election, serving as the central transactional table of the system.

**Attributes of Vote**

| Attribute | Description |
| --- | --- |
| **vote_id** | Unique identifier for each vote record (Primary Key) |
| **voter_id** | Foreign key linking the vote to the voter who cast it (Unique) |
| **candidate_id** | Foreign key linking the vote to the candidate who received it |
| **polling_station_id** | Foreign key linking the vote to the polling station where it was cast |
| **vote_date** | The date on which the vote was cast |
| **is_valid** | A flag indicating whether the vote is valid or invalid |

**Purpose of the Entity**

This entity serves as the core data recording component of the voting system. It captures all voting transactions and enforces the one-voter-one-vote rule through the UNIQUE constraint on voter_id, preventing any form of duplicate voting.

**Relationships**

- One vote is cast by one voter.
- One vote is received by one candidate.
- One vote is recorded at one polling station.

---

### 2.2.7 Admin Entity

The admin entity stores the account information of system administrators who are responsible for managing and overseeing the Electronic Voting System.

**Attributes of Admin**

| Attribute | Description |
| --- | --- |
| **admin_id** | Unique identifier for each administrator (Primary Key) |
| **full_name** | The complete name of the administrator |
| **email** | The administrator's email address (Unique) |
| **username** | The administrator's login username (Unique) |
| **password_hash** | The encrypted password of the administrator's account |
| **role** | The role or access level assigned to the administrator (e.g., manager) |

| | |
|---|---|
| **created_at** | The date on which the administrator account was created |
| **last_login** | The date of the administrator's most recent login |

**Purpose of the Entity**

This entity serves as the access control and user management component for system administrators. It enables secure login, role-based access control, and tracking of administrative activities within the Electronic Voting System.

**Relationships**

- One admin can manage many voters.
- One admin can manage many polling stations.

### 2.3 Relationship Analysis

The following table summarizes all the relationships identified in the Electronic Voting System database:

| Relationship | Entity 1 | Entity 2 | Cardinality | Foreign Key |
|---|---|---|---|---|
| **Belongs to** | Voter | Region | Many-to-One | region_code in Voter |
| **Located in** | Polling Station | Region | Many-to-One | region_code in Polling Station |
| **Nominates** | Party | Candidate | One-to-Many | party_id in Candidate |
| **Casts** | Voter | Vote | One-to-One | voter_id in Vote |
| **Receives** | Candidate | Vote | One-to-Many | candidate_id in Vote |
| **Records votes at** | Polling Station | Vote | One-to-Many | polling_station_id in Vote |
| **Manages** | Admin | Voter | One-to-Many | — |
| **Manages** | Admin | Polling Station | One-to-Many | — |

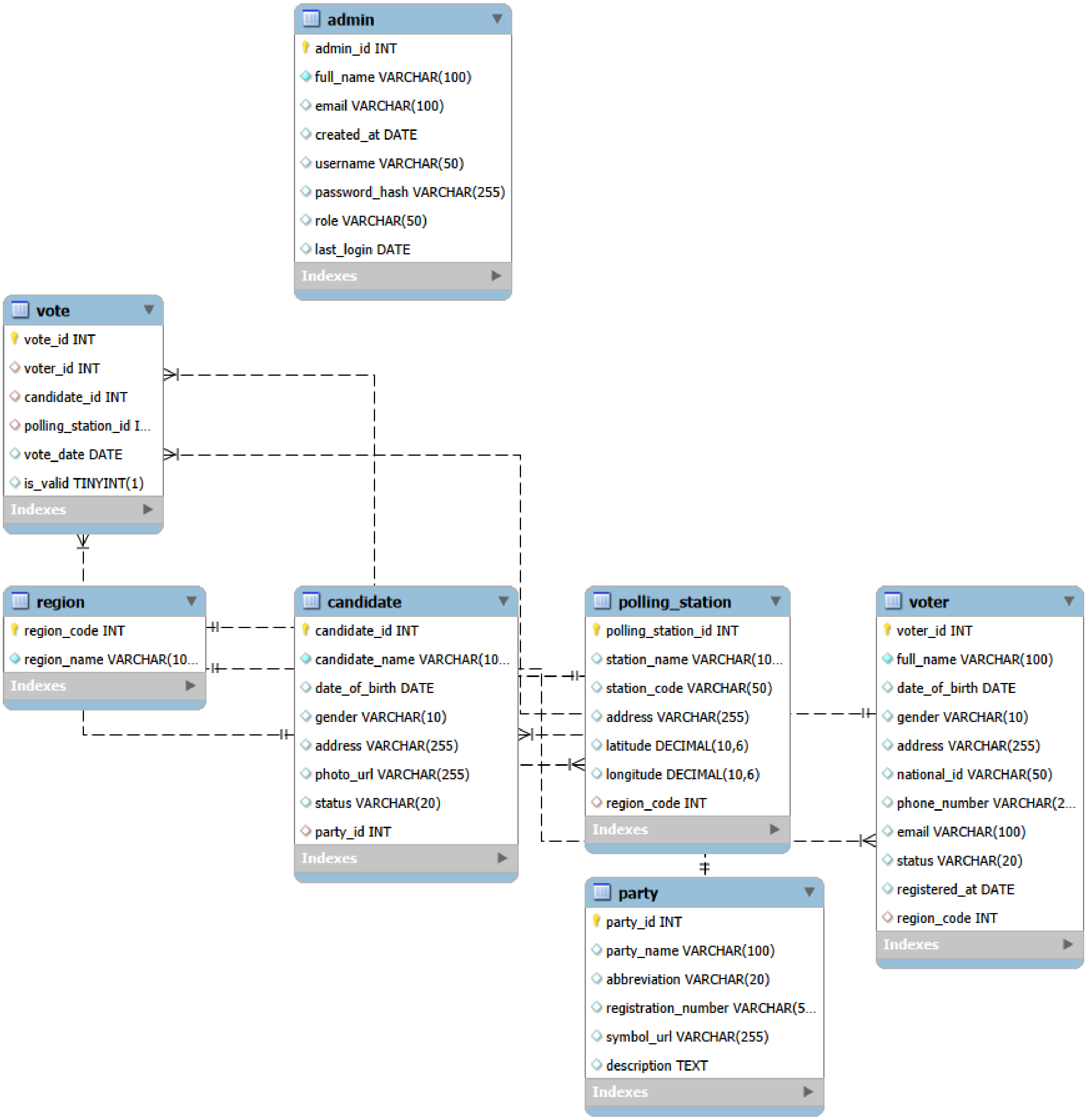
### 2.4 Logical Schema Design

The logical schema design represents the next stage of database design, bridging the gap between the conceptual ER diagram and the physical implementation of the database. While the ER diagram focuses on identifying entities and their relationships at a high level, the logical schema translates those concepts into a more structured and detailed representation by defining the specific tables, attributes, data types, primary keys, foreign keys, and constraints that will govern the database. At this stage, the design is independent of any specific database management system, focusing purely on the logical organization of data.

In the logical schema of the Electronic Voting System, each entity identified in the ER diagram is transformed into a relational table. The attributes of each entity become the columns of their respective tables, and each column is assigned an appropriate data type to ensure that only valid and consistent data is stored. Primary keys are designated to uniquely identify each record within a table, while foreign keys are used to establish and enforce the relationships between tables as identified during the relationship analysis. Additionally, UNIQUE constraints are applied to attributes that must hold distinct values across all records, such as national_id, phone_number, and email in the Voter table, and station_code in the Polling Station table.

The logical schema of the Electronic Voting System consists of seven interrelated tables: Region, Voter, Party, Candidate, Polling Station, Vote, and Admin. These tables are linked through carefully defined foreign key relationships that reflect the real-world interactions of the election process. The schema ensures referential integrity, meaning that no record in a child table can reference a non-existent record in a parent table, thereby maintaining the consistency and reliability of all election data stored in the system.

The following diagram illustrates the complete logical schema design of the Electronic Voting System database, showing all tables, their attributes with data types, and the relationships between them:


admin
admin_id INT
full_name VARCHAR(100)
email VARCHAR(100)
created_at DATE
username VARCHAR(50)
password_hash VARCHAR(255)
role VARCHAR(50)
last_login DATE
Indexes
vote
vote_id INT
voter_id INT
candidate_id INT
polling_station_id I...
vote_date DATE
is_valid TINYINT(1)
Indexes
region
region_code INT
region_name VARCHAR(10...
Indexes
candidate
candidate_id INT
candidate_name VARCHAR(10...
date_of_birth DATE
gender VARCHAR(10)
address VARCHAR(255)
photo_url VARCHAR(255)
status VARCHAR(20)
party_id INT
Indexes
polling_station
polling_station_id INT
station_name VARCHAR(10...
station_code VARCHAR(50)
address VARCHAR(255)
latitude DECIMAL(10,6)
longitude DECIMAL(10,6)
region_code INT
Indexes
voter
voter_id INT
full_name VARCHAR(100)
date_of_birth DATE
gender VARCHAR(10)
address VARCHAR(255)
national_id VARCHAR(50)
phone_number VARCHAR(2...
email VARCHAR(100)
status VARCHAR(20)
registered_at DATE
region_code INT
Indexes
party
party_id INT
party_name VARCHAR(100)
abbreviation VARCHAR(20)
registration_number VARCHAR(5...
symbol_url VARCHAR(255)
description TEXT
Indexes

# CHAPTER 3: IMPLEMENTATION

We developed The Voting System database using MySQL to manage and organize election-related information. The system stores data about regions, voters, political parties, candidates, polling stations, votes, and administrators. The database helps ensure efficient storage, retrieval, and analysis of election data.

## 3.1 Database and Table Creation

**Database Creation**

The database was created using the following SQL commands:

```
CREATE DATABASE Voting_System;

USING Voting_System;
```

**Explanation**

- CREATE DATABASE creates a new database named Voting_System.
- USE selects the database for further operations.

**Table Creation**

Several tables were created to store different types of election information.

We created tables using the command

```
 CREATE TABLE <table name> (

Attributes,

)
```

## 3.2 Data Insertion and Sample Records

Sample data was inserted into all tables using INSERT INTO statements.

**Example:**

```
INSERT INTO region (region_code, region_name) VALUES

(1, 'Addis Ababa'),

(2, 'Oromia'),

(3, 'Amhara');
```

Additional sample records were inserted into:

- voter table
- candidate table
- party table
- polling_station table
- vote table
- admin table

**Explanation**

The inserted sample data was used to test the functionality of the database and simulate a real election environment.

### 3.3 SQL Constraints and Data Integrity

Some SQL constraints used

| constraints | usage |
|---|---|
| **PRIMARY KEY** | Uniquely identifies each record |
| **FOREIGN KEY** | To create relationship |
| **UNIQUE** | Prevent duplicate data |
| **AUTO_INCREAMENT** | Automatically generate IDs |
| **DEFAULT** | To assign default values |

### 3.4 Result Analysis and Vote Counting Query

A SQL query was created to display each candidate, their political party, and the total number of votes they received

```
SELECT

    c.candidate_id,

    c.candidate_name,

    p.party_name,

    COUNT(v.vote_id) AS total_votes

FROM candidate c

LEFT JOIN party p

    ON c.party_id = p.party_id

LEFT JOIN vote v

    ON c.candidate_id = v.candidate_id
```

**Explanation**

- **SELECT** was to retrieve candidate names, party names, and total votes.
- **LEFT JOIN** to connect the candidate, party, and vote tables.
- **COUNT**(v.vote_id) calculated the total votes received by each candidate.
- **GROUP BY** grouped the records according to candidate and party.
- **ORDER BY** TotalVotes DESC sorted the results from highest to lowest votes.

**Result:**

| candidate_id | candidate_name | party_name | total_votes |
|---|---|---|---|
| 1 | Hana M | Prosperity Party | 2 |
| 2 | Samuel K | Unity Party | 1 |

This query helps identify the leading candidate in the election.

### 3.5 MongoDB Implementation

In addition to the relational MySQL database, the Electronic Voting System also implements a NoSQL database using MongoDB. MongoDB stores data as flexible JSON-like documents organized into collections, making it suitable for scenarios that require scalable and schema-flexible data storage. While MySQL handles structured relational data with strict constraints, MongoDB provides an alternative storage approach that complements the relational design. This section presents the MongoDB collections and queries implemented for the Electronic Voting System.

---

#### 3.5.1 MongoDB Collections

MongoDB organizes data into collections, which are equivalent to tables in a relational database. Each document within a collection represents a single record stored in JSON format. The Electronic Voting System implements six MongoDB collections that mirror the structure of the MySQL relational tables.

The following collections were created and populated with sample data as defined in the collections.json file:

**1. regions Collection**

Stores the geographical regions of Ethiopia where voters are registered and polling stations are located.

```
json

{

  "region_code": 1,

  "region_name": "Addis Ababa"

}
```

**2. parties Collection**

Stores information about registered political parties participating in the election.

```json
{
  "party_id": 1,
  "party_name": "Prosperity Party",
  "abbreviation": "PP",
  "registration_number": "REG001",
  "symbol_url": "pp.png",
  "description": "Ruling party"
}
```

**3. polling_stations Collection**

Stores the physical locations where voters cast their votes, including geographic coordinates.

```json
{
  "station_name": "Bole Station",
  "station_code": "BOL001",
  "address": "Bole Road",
  "latitude": 8.9806,
  "longitude": 38.7578,
  "region_code": 1
}
```

**4. voters Collection**

Stores personal information of all registered voters in the system.

```json
{

  "voter_id": 1,

  "full_name": "Abel Tesfaye",

  "date_of_birth": "2000-01-01",

  "gender": "Male",

  "address": "Bole",

  "national_id": "NID001",

  "phone_number": "0911111111",

  "email": "abel@gmail.com",

  "status": "active",

  "registered_at": "2025-01-10",

  "region_code": 1

}
```

**5. candidates Collection**

Stores information about all candidates contesting in the election, including their party affiliation.

```json
{

  "candidate_id": 1,

  "candidate_name": "Candidate A",

  "party_id": 1,

  "status": "active"

}
```

**6. votes Collection**

Stores all vote transactions recorded during the election, linking voters, candidates, and polling stations.

```json
json

{

  "vote_id": 1,

  "voter_id": 1,

  "candidate_id": 1,

  "polling_station_id": 1,

  "vote_date": "2025-05-15",

  "is_valid": true

}
```

**Result**: All six collections were successfully created and populated with sample data representing regions, political parties, polling stations, voters, candidates, and votes in the Electronic Voting System.

---

### 3.5.2 MongoDB Queries

Four MongoDB queries were implemented to demonstrate the core data operations of the Electronic Voting System. These queries cover data insertion, relational lookups, vote aggregation, and data filtering, and are equivalent in function to the SQL operations performed in the MySQL implementation.

---

Query 1 — Insert a New Voter (insertOne)

This query inserts a new voter document into the voters collection. It is equivalent to the SQL INSERT INTO statement used in the MySQL implementation.

```javascript
javascript

db.voters.insertOne({
```

```
  voter_id: 2,

  full_name: "John Doe",

  email: "john@example.com",

  status: "active",

  region_code: 1,

});
```

**Explanation:**

- db.voters.insertOne() inserts a single new document into the voters collection.
- The document contains the voter's ID, full name, email, status, and region code.
- This operation registers a new voter into the MongoDB database.

**Result**: A new voter document for John Doe was successfully inserted into the voters collection with an active status and assigned to region 1 (Addis Ababa).

---

Query 2 — Join Candidates with Party Details ($lookup)

This query uses the MongoDB aggregation pipeline with a $lookup stage to join the candidates collection with the parties collection. It is equivalent to the SQL JOIN operation, combining candidate information with their respective party details.

```
javascript

db.candidates.aggregate([

  {

    $lookup: {

      from: "parties",

      localField: "party_id",

      foreignField: "party_id",

      as: "party_details",
```

```
    },

  },

]);
```

Explanation:

- aggregate() processes the candidates collection through a pipeline of stages.
- $lookup performs a left outer join between the candidates and parties collections, matching documents where the party_id field in candidates equals the party_id field in parties.
- The matched party documents are added to each candidate document as an array field named party_details.
- This query is equivalent to the SQL LEFT JOIN between the candidate and party tables.

**Result**: All candidate documents were returned with their corresponding party details embedded as party_details, successfully linking each candidate to their political party without the need for a separate query.

---

Query 3 — Count Total Votes Per Candidate ($group and $lookup)

This query uses the MongoDB aggregation pipeline to count the total number of votes received by each candidate and sort the results from highest to lowest. It is equivalent to the SQL vote counting query implemented in total_vote.sql using COUNT, GROUP BY, and ORDER BY.

```
javascript

db.votes.aggregate([

  {

    $group: {

      _id: "$candidate_id",

      total_votes: { $sum: 1 },

    },

  },
```

```
  {
    $lookup: {
      from: "candidates",
      localField: "_id",
      foreignField: "candidate_id",
      as: "candidate_info",
    },
  },
  { $unwind: "$candidate_info" },
  {
    $project: {
      candidate_id: "$_id",
      candidate_name: "$candidate_info.candidate_name",
      total_votes: 1,
      _id: 0,
    },
  },
  { $sort: { total_votes: -1 } },
]);
```

**Explanation:**

- $group groups all vote documents by candidate_id and counts the total number of votes each candidate received using $sum: 1.
- $lookup joins the grouped results with the candidates collection to retrieve each candidate's name.
- $unwind flattens the candidate_info array produced by $lookup into individual documents.

- $project shapes the final output to include only the candidate_id, candidate_name, and total_votes fields.
- $sort orders the results by total_votes in descending order, placing the candidate with the most votes at the top.

**Result**: All candidates were returned with their total vote counts sorted from highest to lowest, successfully identifying the leading candidate in the election. This query produces the same result as the SQL vote counting query in total_vote.sql.

---

Query 4 — Find Voters by Region (find)

This query retrieves all voter documents from the voters collection that belong to a specific region. It is equivalent to the SQL SELECT with a WHERE clause used to filter voters by region.

```
javascript

db.voters.find({ region_code: 1 });
```

**Explanation:**

- db.voters.find() retrieves all documents from the voters collection that match the specified filter condition.
- { region_code: 1 } filters the results to return only voters whose region_code equals 1, which corresponds to Addis Ababa.

This query allows the system to generate region-specific voter lists for administrative and reporting purposes.

**Result:** All voter documents registered under region code 1 (Addis Ababa) were successfully retrieved, demonstrating the system's ability to filter and display voters by geographic region.

## 3.6 Input, Output, and Report Design

### 3.6.1 Input Design

Input design focuses on how data is entered into the system. Proper input design helps reduce errors, improve data accuracy, and simplify system usage.

**1.Voter Registration Form**

**Purpose**

Used to register voters into the system.

**Input Table**

| Field Name | Type | Description |
|---|---|---|
| **voter_id** | Integer | Primary Key |
| **full_name** | Text | Voter full name |
| **date_of_birth** | Date | Birth date |
| **gender** | Text | Male or Female |
| **address** | Text | Home address |
| **national_id** | Text | Unique national ID |
| **phone_number** | Text | Contact number |
| **email** | Text | Email address |
| **region_code** | Foreign Key | Region reference |
| **status** | Text | Active/Inactive |

**2.Candidate Registration Form**

**Purpose**

Used to register election candidates.

**Input Table**

| Field Name | Type | Description |
|---|---|---|
| **candidate_id** | Integer | Primary Key |
| **candidate_name** | Text | Candidate name |
| **date_of_birth** | Date | Birth date |

| | | |
|---|---|---|
| **gender** | Text | Gender |
| **address** | Text | Address |
| **party_id** | Foreign Key | Party reference |
| **status** | Text | Candidate status |
| **photo_url** | Text | Candidate photo |

**3.Political Party Form**

**Purpose**

Used to register political parties.

**Input Table**

| Field Name | Type | Description |
|---|---|---|
| **party_id** | Integer | Primary Key |
| **party_name** | Text | Party name |
| **party_abbreviation** | Text | Short name |
| **registration_number** | Text | Unique registration number |
| **description** | Text | Party description |
| **symbol_url** | Text | Party logo |

**4.Polling Station Form**

**Purpose**

Used to register polling stations.

**Input Table**

| Field Name | Type | Description |
|---|---|---|
| **polling_station_id** | Integer | Primary Key |
| **station_name** | Text | Station name |
| **station_code** | Text | Unique code |

| | | |
|---|---|---|
| **address** | Text | Station address |
| **latitude** | Decimal | GPS latitude |
| **longitude** | Decimal | GPS longitude |
| **region_code** | Foreign Key | Region reference |

**5.Voting Form**

**Purpose**

Used by voters to cast votes.

**Input Table**

| Field Name | Type | Description |
|---|---|---|
| **vote_id** | Integer | Primary Key |
| **voter_id** | Foreign Key | Voter reference |
| **party_id** | Foreign Key | Selected party |
| **polling_station_id** | Foreign Key | Polling station |
| **vote_date** | DateTime | Voting time |
| **is_valid** | Boolean | Vote validity |

---

### 3.6.2 Output Design

Output design explains how processed information is displayed to users.

---

**1.Voter Information Output**

Sample Output

| |
|---|
| Voter ID : 1001 |
| Full Name : abebe balcha |
| National ID : ET123456 |

| | |
|---|---|
| Region | : Addis Ababa |
| Status | : Active |

**2 Candidate Information Output**

Sample Output

| |
|---|
| Candidate Name : Kebede mola |
| Political Party: prosperity Party |
| Status : Active |

**3 Election Result Output**

Sample Output

| Party Name | Total Votes | Percentage |
|---|---|---|
| **Party A** | 15,000 | 45% |
| **Party B** | 12,000 | 36% |
| **Party C** | 6,000 | 19% |

**4 Polling Station Output**

Sample Output

| Station Code | Station Name | Region |
|---|---|---|
| **PS001** | Addis Station 01 | Addis Ababa |
| **PS002** | Adama Station 02 | Oromia |

**5 Notification Messages**

Example Messages

✓ Registration Completed Successfully

✓ Vote Submitted Successfully

✕ Invalid Username or Password

✕ Duplicate Vote Detected

**3.6.3 Report Design**

Reports are generated to support election management and monitoring.

**1 Registered Voters Report**

**Purpose**

Displays all registered voters.

Sample Report Table

| Voter ID | Full Name | Region | Status |
|---|---|---|---|
| **1001** | Dawit Aregawi | Addis Ababa | Active |
| **1002** | Abel Tesfaye | Oromia | Active |

**2 Election Result Report**

**Purpose**

Displays election vote results.

---

Sample Report Table

| Party | Votes | Percentage |
|---|---|---|
| **Party A** | 15,000 | 45% |
| **Party B** | 12,000 | 36% |
| **Party C** | 6,000 | 19% |

---

**3 Candidate Report**

Sample Report Table

| Candidate Name | Party | Status |
|---|---|---|
| **Abebe Kebede** | Unity Party | Active |
| **Hana Bekele** | National Party | Active |

---

**4 Polling Station Report**

Sample Report Table

| Station Name | Region | Address |
|---|---|---|
| **Station 01** | Addis Ababa | Bole |
| **Station 02** | Oromia | Adama |

**5 Invalid Vote Report**

Sample Report Table

| Vote ID | Voter ID | Reason |
|---|---|---|
| **5001** | 1002 | Duplicate Vote |
| **5002** | 1045 | Invalid Candidate |

---

**6 Admin Activity Report**

Sample Report Table

| Admin Name | Activity | Date |
|---|---|---|
| **Admin01** | Registered Voter | 2026-05-01 |
| **Admin02** | Updated Polling Station | 2026-05-02 |

---

### 3.7 Implementation Summary

The Electronic Voting System database was successfully implemented using both MySQL and MongoDB. All planned features were developed, tested, and verified to function correctly. The following table summarizes the complete implementation of the system:

| Component | Details | Status |
|---|---|---|
| **MySQL Database** | Voting_System database created | ✓ Completed |
| **MySQL Tables** | 7 tables created — region, voter, party, candidate, polling_station, vote, admin | ✓ Completed |
| **SQL Constraints** | PRIMARY KEY, FOREIGN KEY, UNIQUE, AUTO_INCREMENT, DEFAULT applied across all tables | ✓ Completed |
| **Sample Data Insertion** | Sample records inserted into all 7 tables using INSERT INTO statements | ✓ Completed |
| **Vote Counting Query** | LEFT JOIN query implemented to count total votes per candidate including zero vote candidates | ✓ Completed |
| **MongoDB Collections** | 6 collections created — regions, parties, polling_stations, voters, candidates, votes | ✓ Completed |
| **MongoDB Queries** | 4 queries implemented — insertOne, $lookup, $group with vote counting, and find by region | ✓ Completed |
| **Input Design** | 5 input forms designed — voter registration, candidate registration, party, polling station, and voting form | ✓ Completed |
| **Output Design** | 5 output displays designed — voter info, candidate info, election results, polling station, and notification messages | ✓ Completed |
| **Report Design** | 6 reports designed — registered voters, election results, candidate, polling station, invalid vote, and admin activity | ✓ Completed |

The implementation demonstrates the effective use of relational database principles including primary keys, foreign keys, unique constraints, and SQL joins, alongside NoSQL document storage using MongoDB. Together, both database systems provide a comprehensive, reliable, and efficient foundation for managing election data for the National Election Board of Ethiopia.

# CHAPTER FOUR NORMALIZATION

Normalization is the process of organizing a relational database to reduce data redundancy, eliminate anomalies, and improve overall data integrity. It involves decomposing large, poorly structured tables into smaller, well-defined relations by applying a series of rules known as normal forms. Each normal form builds upon the previous one, progressively eliminating different types of data dependencies and redundancies until the database reaches an optimal structure.

In the context of the Electronic Voting System developed for the National Election Board of Ethiopia, normalization is essential to ensure that voter data, candidate information, party details, polling station records, and vote transactions are stored efficiently and consistently. Any redundancy or anomaly in election data could lead to incorrect vote counts, duplicate voter records, or inconsistent results, all of which would undermine the integrity of the election process.

In this project, the database is normalized step by step from Unnormalized Form (UNF) through First Normal Form (1NF), Second Normal Form (2NF), Third Normal Form (3NF), and finally to Boyce-Codd Normal Form (BCNF). Each stage is explained in detail, showing the functional dependencies, the changes applied, and the resulting table structures.

## 4.1 Problems in UNF

Before normalization, all election data existed in a single unnormalized flat table. This table contained repeated and mixed data from voters, candidates, parties, polling stations, and votes all stored together, leading to significant redundancy and inconsistency. The following table represents the Unnormalized Form (UNF) of the voting system data:

| voter_ id | voter_ name | national_ id | region | candidate_ name | party_ name | party_ abbr | station_ name | station_ code | vote_ date | is_ valid |
|---|---|---|---|---|---|---|---|---|---|---|
| **1** | Abel | NID001 | Addis Ababa | Hana M | Ezema | EZ | Bole | BOL01 | 06/01 | Yes |
| **2** | Kidus | NID002 | Afar | Hana M | Unity | UP | Bole | BOL01 | 06/01 | Yes |
| **3** | Selam | NID003 | Addis Ababa | Samuel K | Ezema | EZ | Piassa | PIA01 | 06/01 | Yes |

- Data redundancy exists.
- Candidate, voter, party, and region information are repeated many times.
- Update anomalies may occur.
- Insert and delete anomalies may occur.
- Difficult to maintain consistency.

Therefore, normalization is required.

## 4.2 First Normal Form (1NF)

A relation is in First Normal Form when:

- All attributes contain atomic values.
- There are no repeating groups.
- Each record is unique.

**1NF Relations**

### Region

| | | |
|---|---|---|
| region_code (PK) | region_name | |

### Voter

| | | | | |
|---|---|---|---|---|
| voter_id (PK) | full_name | date_of_birth | gender | address |
| national_id | phone_number | email | status | registered_at |
| region_code (FK) | | | | |

### Polling_station

| | | | | | |
|---|---|---|---|---|---|
| polling_station_id (PK) | station_name | station_code | address | latitude | longitude |
| region_code (FK) | | | | | |

### Party

| | |
|---|---|
| party_id (PK) | party_name |

| | |
|---|---|
| party_abbreviation | registration_number |
| symbol_url | description |

### Candidate

| | | | | |
|---|---|---|---|---|
| candidate_id (PK) | candidate_name | date_of_birth | gender | address |
| photo_url | status | party_id (FK) | | |

### Admin

| | | | | | | |
|---|---|---|---|---|---|---|
| admin_id (PK) | full_name | email | username | password_hash | role | last_login |
| created_at | | | | | | |

### Vote

| | |
|---|---|
| vote_id (PK) | vote_date |
| is_valid | voter_id (FK) |
| party_id (FK) | polling_station_id (FK) |

The above relations satisfy First Normal Form because all attributes contain atomic values with no repeating groups. For example, in the VOTER table each attribute such as full_name, national_id, phone_number, and email holds a single indivisible value per record. Similarly, the VOTE table separates each vote transaction into its own record rather than storing multiple votes in a single field. Each table has a designated primary key that uniquely identifies every record, and no table contains nested or multi-valued attributes. Therefore, all seven relations are confirmed to be in First Normal Form.

At this stage:

- All attributes are atomic.
- Repeating groups are removed.
- Each table has a primary key.

However, partial dependency and transitive dependency may still exist.

## 4.3 Second Normal Form (2NF)

A relation is in Second Normal Form when:

- It is already in 1NF.

- No partial dependency exists.
- Non-key attributes depend on the whole primary key.

In this project, most tables already use single-attribute primary keys. Therefore, partial dependency is automatically removed.

**Function Dependency**

### Region

region_code → region_name

### Voter

- voter_id → full_name, date_of_birth, gender, address, national_id, phone_number, email, status, registered_at, region_code
- national_id → voter_id
- email → voter_id
- phone_number → voter_id

### Polling_Station

- polling_station_id → station_name, station_code, address, latitude, longitude, region_code
- station_code → polling_station_id

### Party

- party_id → party_name, party_abbreviation, registration_number, symbol_url, description
- party_abbreviation → party_id
- registration_number → party_id

### Candidate

candidate_id → candidate_name, date_of_birth, gender, address, photo_url, status, party_id

### Admin

- admin_id → full_name, email, username, password_hash, role, last_login, created_at
- email → admin_id
- username → admin_id

### Vote

- vote_id → vote_date, is_valid, voter_id, party_id, polling_station_id

Since all non-key attributes depend fully on the primary key, the schema is in 2NF.

## 4.4 Third Normal Form (3NF)

A relation is in Third Normal Form when:

- It is already in 2NF.
- No transitive dependency exists.
- Non-key attributes do not depend on other non-key attributes.

**Checking Transitive Dependencies**

#### REGION

No transitive dependency exists.

#### VOTER

All attributes depend directly on voter_id.

No transitive dependency exists.

#### POLLING_STATION

All attributes depend directly on polling_station_id.

No transitive dependency exists.

#### PARTY

All attributes depend directly on party_id.

No transitive dependency exists.

#### CANDIDATE

All attributes depend directly on candidate_id.

No transitive dependency exists.

### ADMIN

All attributes depend directly on admin_id.

No transitive dependency exists.

### VOTE

All attributes depend directly on vote_id.

No transitive dependency exists.

Therefore, the database is in 3NF.

---

## Boyce-Codd Normal Form (BCNF)

A relation is in BCNF when:

- It is already in 3NF.
- Every determinant is a candidate key.

## 4.5 BCNF Verification

### REGION

Determinant:

- region_code → region_name

region_code is a candidate key. Therefore, REGION is in BCNF.

### VOTER

Determinants:

- voter_id → all voter attributes
- national_id → voter_id
- email → voter_id
- phone_number → voter_id

All determinants are candidate keys because national_id, email, and phone_number are unique. Therefore, VOTER is in BCNF.

### POLLING_STATION

Determinants:

- polling_station_id → all attributes
- station_code → polling_station_id

Both are candidate keys.
Therefore, POLLING_STATION is in BCNF.

### PARTY

Determinants:

- party_id → all attributes
- party_abbreviation → party_id
- registration_number → party_id

All determinants are candidate keys. Therefore, PARTY is in BCNF.

### CANDIDATE

Determinant:

- candidate_id → all attributes

candidate_id is a candidate key. Therefore, CANDIDATE is in BCNF.

### ADMIN

Determinants:

- admin_id → all attributes
- email → admin_id
- username → admin_id

All determinants are candidate keys. Therefore, ADMIN is in BCNF.

### VOTE

Determinant:

- vote_id → vote_date, is_valid, voter_id, candidate_id, polling_station_id

vote_id is a candidate key. Therefore, VOTE is in BCNF.

---

## 4.6 Final BCNF Relations

### REGION

- region_code (PK)
- region_name

### VOTER

- voter_id (PK)
- full_name
- date_of_birth
- gender
- address
- national_id (UNIQUE)
- phone_number(UNIQUE)
- email
- status
- registered_at
- • region_code (FK)

### POLLING_STATION

- polling_station_id (PK)
- station_name
- station_code (UNIQUE)
- address
- latitude
- longitude
- region_code (FK)

### PARTY

- party_id (PK)
- party_name
- party_abbreviation (UNIQUE)
- description
- registration_number (UNIQUE)
- symbol_url

### CANDIDATE

- candidate_id (PK)
- candidate_name
- date_of_birth
- gender
- address
- photo_url
- status
- party_id (FK)

### ADMIN

- admin_id (PK)
- email (UNIQUE)
- username (UNIQUE)
- password_hash
- full_name
- role
- last_login
- created_at

### VOTE

- vote_id (PK)
- vote_date
- is_valid
- voter_id (FK)
- party_id (FK)
- polling_station_id (FK)

---

### 4.7 Conclusion

The Voting Management System database was successfully normalized from Unnormalized Form (UNF) through all stages up to Boyce-Codd Normal Form (BCNF). The normalization process was applied systematically across all seven tables — Region, Voter, Party, Candidate, Polling Station, Vote, and Admin. During the normalization process the following improvements were achieved:

- Redundant data was eliminated by separating repeated information such as region names and party details into their own dedicated tables.
- Data consistency was improved by ensuring that each piece of information is stored in only one place, so updates only need to be made once.

- Insert, update, and delete anomalies were minimized through proper table decomposition and the use of foreign key relationships.
- Data integrity was strengthened through the enforcement of primary keys, foreign keys, and unique constraints across all tables.
- The one-voter-one-vote rule was enforced at the database level through the UNIQUE constraint on voter_id in the VOTE table, directly supporting the core requirement of a fair election system.

The final normalized database structure is efficient, scalable, and fully suitable for implementation in both MySQL and MongoDB systems, providing a reliable foundation for managing election data for the National Election Board of Ethiopia.

---

## CONCLUSION

This report has presented the complete design, implementation, and normalization of the Electronic Voting System database developed for the National Election Board of Ethiopia. The system was developed to address the critical inefficiencies of the existing manual electoral process, which suffers from long waiting times, human errors, duplicate voting risks, high operational costs, and limited data security. By transitioning to a computerized database-driven system, the National Election Board of Ethiopia can significantly improve the efficiency, accuracy, transparency, and security of the election process.

The database design phase established a solid conceptual and logical foundation for the system. Seven core entities were identified — Region, Voter, Party, Candidate, Polling Station, Vote, and Admin — and their relationships were carefully mapped through a comprehensive Entity-Relationship Diagram. The logical schema was then developed to translate these conceptual entities into well-structured relational tables with clearly defined attributes, data types, primary keys, foreign keys, and unique constraints. The one-voter-one-vote rule, which is the most fundamental requirement of any fair election system, was enforced directly at the database level through a UNIQUE constraint on the voter_id attribute in the Vote table, eliminating any possibility of duplicate voting through the system.

The implementation phase demonstrated the successful physical realization of the database design using both MySQL and MongoDB. All seven MySQL tables were created with proper constraints and populated with sample data that simulates a real election environment. The vote counting query implemented using LEFT JOIN, COUNT, GROUP BY, and ORDER BY operations successfully produced accurate election results, including candidates who received zero votes. In parallel, six MongoDB collections were created and four operational queries were implemented, demonstrating the system's capability in a NoSQL environment. The Input, Output, and Report Design sections further illustrated how data flows through the system from entry to display, covering voter registration, candidate management, polling station administration, and election result reporting.

The normalization chapter confirmed that the database schema satisfies all requirements up to Boyce-Codd Normal Form (BCNF). By systematically eliminating data redundancy, partial dependencies, and transitive dependencies across all seven tables, the database achieves a high level of data integrity, consistency, and maintainability. The normalization process ensures that the system can reliably store and retrieve election data without the anomalies that plague manual or poorly designed systems.

In conclusion, the Electronic Voting System database provides a comprehensive, efficient, and secure foundation for modernizing election management in Ethiopia. It successfully meets all the specific objectives defined at the beginning of this project, including secure voter management, duplicate vote prevention, automated vote counting, fast result generation, and reduced operational complexity. The system represents a significant step toward strengthening democratic governance in Ethiopia by ensuring that elections are conducted in a manner that is not only free and fair but also demonstrably efficient, accurate, and transparent in the digital age.

**REFERENCES**

[1] R. Elmasri and S. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ: Pearson, 2016.

[2] C. J. Date, An Introduction to Database Systems, 8th ed. Boston, MA: Addison-Wesley, 2004.

[3] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY: McGraw-Hill, 2019.

[4] National Election Board of Ethiopia (NEBE), "About NEBE," National Election Board of Ethiopia, Addis Ababa, Ethiopia, 2021. [Online]. Available: https://www.nebe.org.et

[5] Oracle Corporation, "MySQL 8.0 Reference Manual," MySQL Documentation, 2023. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[6] MongoDB, Inc., "MongoDB Manual," MongoDB Documentation, 2023. [Online]. Available: https://www.mongodb.com/docs/manual/

[7] P. Rob and C. Coronel, Database Systems: Design, Implementation, and Management, 10th ed. Boston, MA: Cengage Learning, 2013.

[8] T. Connolly and C. Begg, Database Systems: A Practical Approach to Design, Implementation, and Management, 6th ed. Harlow, UK: Pearson, 2015.

[9] W. J. Premerlani and M. R. Blaha, "An approach for reverse engineering of relational databases," Communications of the ACM, vol. 37, no. 5, pp. 42–49, May 1994.

[10] K. C. Laudon and J. P. Laudon, Management Information Systems: Managing the Digital Firm, 16th ed. Hoboken, NJ: Pearson, 2020.

[11] Draw.io (Diagrams.net), "Draw.io Diagram Tool," Diagrams.net, 2023. [Online]. Available: https://www.diagrams.net

[12] GitHub, Inc., "GitHub Documentation," GitHub Docs, 2023. [Online]. Available: https://docs.github.com

[13] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377–387, Jun. 1970.

[14] W. Kent, "A simple guide to five normal forms in relational database theory," Communications of the ACM, vol. 26, no. 2, pp. 120–125, Feb. 1983.

[15] International Institute for Democracy and Electoral Assistance (IDEA), "Electronic Voting," International IDEA, Stockholm, Sweden, 2011. [Online]. Available: https://www.idea.int